Multiplicación de matrices grandes

La multiplicación de matrices consiste en calcular la multiplicación de los elementos C_{ij}, con base en el producto de los elementos de la fila $i_{ésima}$ de una matriz A, por los elementos de la columna $j_{ésima}$ de una matriz B. Si A es una matriz de tamaño m x n y B es una matriz de tamaño n x p, el tamaño del producto de A x B, genera una matriz C de tamaño m x p. La siguiente corresponde a la estructura de una multiplicación de dos matrices.

$$A = \begin{pmatrix} a11 & a12 \\ a21 & a22 \\ a31 & a32 \end{pmatrix} \quad * \quad B = \begin{pmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \end{pmatrix} \quad = \quad C\begin{pmatrix} c11 & c12 & c13 \\ c21 & c22 & c23 \\ c32 & c32 & c33 \end{pmatrix}$$

La matriz A, es una matriz de tamaño 3 x 2 y la matriz B es una matriz de tamaño 2 x 3. La matriz resultante C, es una matriz de tamaño 3 x 3. También se debe tener en cuenta que para que la multiplicación se lleve a cabo, el número de columnas de la primera matriz debe ser igual al número de filas de la segunda columna. Cada elemento C_{ij} está definido por la siguiente expresión:

$$Cij \ = \sum_{k=1}^{n} Aik * Bkj$$

A continuación, se presentan implementaciones iterativas y divide y vencerás para la multiplicación de matrices.

- **Multiplicación tradicional de matrices**

La multiplicación tradicional de matrices se fundamenta en la implementación de tres ciclos anidados. El ciclo más interno es el responsable de ubicar en la matriz resultado el elemento del producto mediante la operación `auxiliar += (A[i][k] * B[k][j]).` En la variable `auxiliar`, se almacena el producto y posteriormente se asigna mediante la expresión `matriz[i][j] = auxiliar.` Una primera implementación del algoritmo de multiplicación se presenta a continuación:

```
public int[][] naivStandard(int [][] A, int [][] B){

      int [][] matriz = new int [A.length][B[0].length];
      int auxiliar;
      if (A[0].length == B.length) {
            for (int i = 0; i < A.length; i++)
            {
                  for (int j = 0; j < B[0].length; j++)
                  {
                        auxiliar = 0;
                        for (int k = 0; k < B.length; k++) {
                              auxiliar += (A[i][k] * B[k][j]);
                        }
                        matriz[i][j] = auxiliar;
                  }
            }
      }
      return matriz;
}
```

Una segunda implementación de la multiplicación de matrices usa directamente la posición `matriz [i][j]` para hacer la asignación del producto de la matriz mediante la expresión `matriz [i][j]+= (A[i][k]*B[k][j])`. También se utilizan tres ciclos anidados.

```
public int[][] naivStandard2(int [][] A, int [][] B)
{
      int [][] matriz = new int [A.length][B[0].length];

      if (A[0].length == B.length)
      {
            for (int i = 0; i < A.length; i++)
            {
                  for (int j = 0; j < B[0].length; j++)
                  {
                        matriz [i][j] = 0;
                        for (int k = 0; k < B.length; k++)
                        {
                              matriz [i][j]+= (A[i][k]*B[k][j]);
                        }
                  }
            }
      }
      return matriz;
}
```

- **Multiplicación por Kahan**

En la publicación de (Kahan, 1965) se plantea una mejora para la estabilidad del proceso de multiplicación mediante la optimización del siguiente computo:

$$\sum_{i=1}^{n} xi$$

La anterior expresión puede ser implementada en Java de la siguiente manera:

```
int suma = 0;
for( int i = 1; i <= n; i++)
{
      suma = suma + x[i];
}
```

Con base en lo propuesto por (Kahan, 1965), el anterior ciclo puede ser expresado mediante:

```
suma = 0;
error = 0;
for (int k = 0; k < B.length; k++)
{
      error = error + A[i][k] * B[k][j];
      t = suma + error;
      error = (suma - t) + error;
      suma = t;
}
```

La implementación recibe las dos matrices y se fundamenta en tres ciclos anidados. En el ciclo más interno se muestra el código con la optimización planteada para gestionar internamente la matriz.

```
public double[][] Kahan(double[][] A, double[][] B)
{
    int filas = A.length;
    int columnas = B[0].length;
    int n = B.length;

    double[][] matriz = new double[filas][columnas];

    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            double suma = 0.0;
            double error = 0.0;
            for (int k = 0; k < n; k++) {
                double producto = A[i][k] * B[k][j];
                double y = producto - error;
                double t = suma + y;
                error = (t - suma) - y;
                suma = t;
            }
            matriz[i][j] = suma;
        }
    }
    return matriz;
}
```

- **Multiplicación mediante desenvolvimiento de ciclos**

El desenvolvimiento de ciclos es una técnica de optimización consiste en desenvolver el cuerpo del ciclo dos o más veces incrementando los saltos de ciclo, con el fin de mejorar el reúso de registros y para minimizar la cantidad de repeticiones y de instrucciones.

A continuación, se muestra como las operaciones incluyen saltos que abarcan dos posiciones a nivel de la fila y la columna de cada matriz. Si se cumple el condicional `if (B.length % 2 == 0)`, significa que la matriz B tiene una cantidad de filas par. En ese bloque se cuenta con tres ciclos anidados y el ciclo más interno es el ciclo que realiza la operación de desenvolvimiento del bucle mediante la instrucción `A[i][k]*B[k][j]+A[i][k+1]*B[k+1][j]`, la cual toma para la primera iteración el producto de a11*b11+a12*b21. En la siguiente estructura se observa cómo se toman pares de valores.

$$A = \begin{bmatrix} a11 & a12 & a13 & a14 \\ a21 & a22 & a23 & a24 \\ a31 & a32 & a33 & a34 \\ a41 & a42 & a43 & a44 \end{bmatrix} \qquad B = \begin{bmatrix} b11 & b12 & b13 & b14 \\ b21 & b22 & b23 & b24 \\ b31 & b32 & b33 & b34 \\ b41 & b42 & b43 & b44 \end{bmatrix}$$

La siguiente es la implementación para multiplicar de matrices con desenvolvimiento de ciclos.

```
public int[][] naivLoopUnrollingtwo(int A[][], int B[][] )
{
	int [][] matriz = new int [A.length][B[0].length];
	int aux, i,j,k;
	if (B.length % 2 == 0)
	{
		for (i = 0; i < A.length; i++)
		{
			for (j = 0; j < B[0].length; j++)
			{
				aux = 0;
				for (k = 0; k < B.length; k += 2)
				{
					aux+=A[i][k]*B[k][j]+A[i][k+1]*B[k+1][j];
				}
				matriz[i][j] = aux;
			}
		}
	}
	else
	{
		int PP = B.length - 1;
		for (i = 0; i < A.length; i++)
		{
			for (j = 0; j < B[0].length; j++)
			{
				aux = 0;
				for (k = 0; k < PP; k += 2)
				{
					aux+=A[i][k]*B[k][j]+A[i][k+1]*B[k+1][j];
				}
				matriz[i][j] = aux + A[i][PP] * B[PP][j];
			}
		}
	}
		return matriz;
}
```

- **Multiplicación por Strassen**

El método de Strassen utiliza la estrategia divide y vencerás para la multiplicación de matrices. Su uso y aplicación se da con frecuencia en matrices muy grandes y su repercusión fundamentalmente ha sido a nivel teórico. La fundamentación de la multiplicación por este método parte de dos matrices A y B, de tamaño n x n, donde $n = 2^k$.

$$A = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix} \qquad B = \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix}$$

Se observa que son 7 multiplicaciones y 4 sumas que se deben realizar para obtener el resultado, lo cual corresponde al mismo número de multiplicaciones que se aplican para aplicar el método tradicional de multiplicación. El método de Strassen, reduce la cantidad de multiplicaciones de la siguiente manera:

$M_1 = (a_{1,1} + a_{2,2}) * (b_{1,1} + b_{2,2})$
$M_2 = (a_{2,1} + a_{2,2}) * b_{1,1}$
$M_3 = a_{1,1} * (b_{1,2} - b_{2,2})$
$M_4 = a_{2,2} * (b_{2,1} – b_{1,1})$
$M_5 = (a_{1,1} + a_{1,2}) * b_{2,2}$
$M_6 = (a_{2,1} - a_{1,1}) * (b_{1,1} + b_{1,2})$
$M_7 = (a_{1,2} - a_{2,2}) * (b_{2,1} + b_{2,2})$

Para expresar la solución en términos de la matriz C, se expresa cada término C_{ij} en términos de M_i, de la siguiente manera:

$C_{1,1} = M_1 + M_4 – M_5 + M_7$
$C_{1,2} = M_3 + M_5$
$C_{2,1} = M_2 + M_4$
$C_{2,2} = M_1 – M_2 + M_3 + M_6$

La matriz resultante C, tendrá la siguiente estructura:

$$C = \begin{bmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{bmatrix}$$

Para el caso base se establece entonces la siguiente estructura, la cual relacionada de forma directa con la expresión matemática.

```
m1 = ((A[0][0] + A[1][1]) * (B[0][0] + B[1][1]));
m2 = ((A[1][0] + A[1][1]) * B[0][0]);
m3 = (A[0][0] * (B[0][1] - B[1][1]));
m4 = (A[1][1] * (B[1][0] - B[0][0]));
m5 = ((A[0][0] + A[0][1]) * B[1][1]);
m6 = ((A[1][0] - A[0][0]) * (B[0][0] + B[0][1]));
m7 = ((A[0][1] - A[1][1]) * (B[1][0] + B[1][1]));
matrizResultados[0][0] = (m1 + m4 - m5 + m7);
matrizResultados[0][1] = (m3 + m5);
matrizResultados[1][0] = (m2 + m4);
matrizResultados[1][1] = (m1 + m3 - m2 + m6);
return matrizResultados;
```

A continuación, se presenta un ejemplo de la multiplicación por Strassen para dos matrices de tamaño 4 x 4. El ejemplo permite entender cómo funciona cuando la matriz llega al caso base tanto para la matriz A y B, y se pueden organizar de acuerdo con los siguientes cuadrantes:

$$A = \begin{bmatrix} 1 & 1 & 0 & 4 \\ 0 & 6 & 8 & 3 \\ 4 & 1 & 1 & 2 \\ 1 & 3 & 4 & 0 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 4 & 2 & 2 \\ 4 & 1 & 3 & 2 \\ 7 & 1 & 1 & 3 \\ 1 & 0 & 5 & 1 \end{bmatrix}$$

Se define cada una de las submatrices para la matriz A.

$$A11 = \begin{bmatrix} 1 & 1 \\ 0 & 6 \end{bmatrix} \qquad A12 = \begin{bmatrix} 0 & 4 \\ 8 & 3 \end{bmatrix} \qquad A21 = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \qquad A22 = \begin{bmatrix} 1 & 2 \\ 4 & 0 \end{bmatrix}$$

Se define cada una de las submatrices para la matriz B.

$$B11 = \begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \qquad B12 = \begin{bmatrix} 2 & 2 \\ 3 & 2 \end{bmatrix} \qquad B21 = \begin{bmatrix} 7 & 1 \\ 1 & 0 \end{bmatrix} \qquad B22 = \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$$

Se calcula $M_1 = (a_{1,1} + a_{2,2}) * (b_{1,1} + b_{2,2})$

$$M1 = \begin{bmatrix} 1 & 1 \\ 0 & 6 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 4 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$$

$$M1 = \begin{bmatrix} 31 & 20 \\ 62 & 40 \end{bmatrix}$$

Se calcula $M_2 = (a_{2,1} + a_{2,2}) * b_{1,1}$

$$M2 = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 4 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix}$$

$$M2 = \begin{bmatrix} 17 & 23 \\ 17 & 23 \end{bmatrix}$$

Se calcula $M_3 = a_{1,1} * (b_{1,2} - b_{2,2})$

$$M3 = \begin{bmatrix} 1 & 1 \\ 0 & 6 \end{bmatrix} * \begin{bmatrix} 2 & 2 \\ 3 & 2 \end{bmatrix} - \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$$

$$M3 = \begin{bmatrix} -1 & 0 \\ -12 & 6 \end{bmatrix}$$

Se calcula $M_4 = a_{2,2} * (b_{2,1} - b_{1,1})$

$$M4 = \begin{bmatrix} 1 & 2 \\ 4 & 0 \end{bmatrix} * \begin{bmatrix} 7 & 1 \\ 1 & 0 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 0 & 6 \end{bmatrix}$$

$$M4 = \begin{bmatrix} 0 & -5 \\ 24 & -12 \end{bmatrix}$$

Se calcula $M_5 = (a_{1,1} + a_{1,2}) * b_{2,2}$

$$M5 = \begin{bmatrix} 1 & 1 \\ 0 & 6 \end{bmatrix} + \begin{bmatrix} 0 & 4 \\ 8 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$$

$$M5 = \begin{bmatrix} 26 & 8 \\ 53 & 33 \end{bmatrix}$$

Se calcula $M_6 = (a_{2,1} - a_{1,1}) * (b_{1,1} + b_{1,2})$

$$M6 = \begin{bmatrix} 1 & 1 \\ 0 & 6 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 4 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$$

$$M6 = \begin{bmatrix} 31 & 20 \\ 62 & 40 \end{bmatrix}$$

Se calcula $M_7 = (a_{1,2} - a_{2,2}) * (b_{2,1} + b_{2,2})$

$$M7 = \begin{bmatrix} 0 & 4 \\ 8 & 3 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 4 & 0 \end{bmatrix} * \begin{bmatrix} 7 & 1 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$$

$$M7 = \begin{bmatrix} 4 & -2 \\ 50 & 19 \end{bmatrix}$$

Con base en los resultados obtenidos para las matrices M1, M2, M3, M4, M5, M6 y M7.

Se calcula $C_{1,1} = M_1 + M_4 - M_5 + M_7$

$$C11 = \begin{bmatrix} 9 & 5 \\ 83 & 14 \end{bmatrix}$$

Se calcula $C_{1,2} = M_3 + M_5$

$$C12 = \begin{bmatrix} 25 & 8 \\ 41 & 39 \end{bmatrix}$$

Se calcula $C_{2,1} = M_2 + M_4$

$$C21 = \begin{bmatrix} 17 & 18 \\ 41 & 11 \end{bmatrix}$$

Se calcula $C_{2,2} = M_1 - M_2 + M_3 + M_6$

$$C22 = \begin{bmatrix} 22 & 15 \\ 15 & 20 \end{bmatrix}$$

El resultado de la multiplicación entre la matriz A y la matriz B, es el siguiente:

$$C = \begin{bmatrix} 9 & 5 & 25 & 8 \\ 83 & 14 & 41 & 39 \\ 17 & 18 & 22 & 15 \\ 41 & 11 & 15 & 20 \end{bmatrix}$$

A continuación, se presenta la implementación del algoritmo de multiplicación de Strassen.

```
public int[][] multiplicarStrassen(int[][] A,int[][] B)
{
      int m1,m2,m3,m4,m5,m6,m7;
      int matrizResultados[][] = new int [A.length][B.length];

      if(A.length == 2)
      {
            m1 = ((A[0][0] + A[1][1]) * (B[0][0] + B[1][1]));
            m2 = ((A[1][0] + A[1][1]) * B[0][0]);
            m3 = (A[0][0] * (B[0][1] - B[1][1]));
            m4 = (A[1][1] * (B[1][0] - B[0][0]));
            m5 = ((A[0][0] + A[0][1]) * B[1][1]);
            m6 = ((A[1][0] - A[0][0]) * (B[0][0] + B[0][1]));
            m7 = ((A[0][1] - A[1][1]) * (B[1][0] + B[1][1]));
```

```
        matrizResultados[0][0] = (m1 + m4 - m5 + m7);
        matrizResultados[0][1] = (m3 + m5);
        matrizResultados[1][0] = (m2 + m4);
        matrizResultados[1][1] = (m1 + m3 - m2 + m6);
        return matrizResultados;
    }
    else
    {
        int n=A.length/2;
        int A00[][] = new int [n][n];
        int A01[][] = new int [n][n];
        int A10[][] = new int [n][n];
        int A11[][] = new int [n][n];
        int B00[][] = new int [n][n];
        int B01[][] = new int [n][n];
        int B10[][] = new int [n][n];
        int B11[][] = new int [n][n];

        for(int i=0; i<n; i++)
        {
            for(int j=0; j<n; j++)
            {
                A00[i][j] = A[i][j];
                B00[i][j] = B[i][j];
            }
        }
        for(int i=0; i<n; i++)
        {
            for(int k=0,j=n; j<A.length; j++, k++)
            {
                A01[i][k] = A[i][j];
                B01[i][k] = B[i][j];
            }
        }
        for(int k=0,i=n; i<A.length; i++, k++)
        {
            for(int j=0; j<n;j++)
            {
                A10[k][j] = A[i][j];
                B10[k][j] = B[i][j];
            }
        }


        for(int k=0, i=n;i<A.length; i++, k++)
        {
            for(int c=0,j=n; j<A.length; j++, c++)
            {
                A11[k][c] = A[i][j];
                B11[k][c] = B[i][j];
            }
        }
```

```
            // En este apartado se invocan las llamadas recursivas para
            dividir el conjunto original de los datos de cada matriz

            int Au1[][]= multiplicar(sumar(A00, A11),sumar(B00,B11));
            int Au2[][]= multiplicar(sumar(A10, A11), B00);
            int Au3[][]= multiplicar(A00,restar(B01,B11));
            int Au4[][]= multiplicar(A11,restar(B10,B00));
            int Au5[][]= multiplicar(sumar(A00, A01),B11);
            int Au6[][]= multiplicar(restar(A10,A00),sumar(B00, B01));
            int Au7[][]= multiplicar(restar(A01,A11),sumar(B10, B11));

            int Cuadrante00[][]=sumar(restar(sumar(Au1,Au4),Au5),Au7);
            int Cuadrante01[][]= sumar(Au3, Au5);
            int Cuadrante10[][]= sumar(Au2, Au4);
            int Cuadrante11[][]=sumar(restar(sumar(Au1,Au3),Au2),Au6);

            for(int i=0; i<n; i++)
            {
                  for(int j=0; j<n; j++)
                  {
                        matrizResultados[i][j] = Cuadrante00[i][j];
                  }
            }
            for(int i=0; i<n; i++)
            {
                  for(int k=0,j=n; j<A.length; j++, k++)
                  {
                        matrizResultados[i][j] = Cuadrante01[i][k];
                  }
            }
            for(int k=0,i=n; i<A.length; i++, k++)
            {
                  for(int j=0; j<n ; j++)
                  {
                        matrizResultados[i][j] = Cuadrante10[k][j];
                  }
            }
            for(int k=0, i=n;i<A.length; i++, k++)
            {
                  for(int c=0,j=n; j<A.length; j++, c++)
                  {
                        matrizResultados[i][j] = Cuadrante11[k][c];
                  }
            }
      }
            return matrizResultados;
}
```

El método `sumar` es el responsable de retornar la suma de dos matrices.

```
public int[][] sumar (int matriz1[][], int matriz2[][])
{
      int suma[][] = new int [matriz1.length][matriz1.length];

      for(int i=0; i<matriz1.length; i++) {
            for(int j=0; j<matriz1.length; j++)
            {
                  suma[i][j] = (int)(matriz1[i][j] + matriz2[i][j]);
            }
      }
      return suma;
}
```

El método `restar` es el responsable de retornar la resta de dos matrices.

```
public int[][] restar(int matriz1[][], int matriz2[][])
{
      int resta[][] = new int [matriz1.length][matriz1.length];

      for(int i=0; i<matriz1.length; i++) {
            for(int j=0; j<matriz1.length; j++) {
                  resta[i][j] = (int)(matriz1[i][j] - matriz2[i][j]);
            }
      }
      return resta;
}
```

- **Actividad**

Dado el siguiente algoritmo, describa su funcionamiento.

```
public int[][] naivLoopUnrollingFour(int[][] A, int[][] B)
{
    int[][] C = new int[A.length][B[0].length];
    int aux, i, j, k;
    int n = A.length, p = B.length, m = B[0].length;

    if (p % 4 == 0) {
        for (i = 0; i < n; i++) {
            for (j = 0; j < m; j++) {
                aux = 0;
                for (k = 0; k < p; k += 4) {
                    aux += A[i][k]*B[k][j] + A[i][k+1]*B[k+1][j] +
                           A[i][k+2]*B[k+2][j] + A[i][k+3]*B[k+3][j];
                }
                C[i][j] = aux;
            }
        }
    }
```

```
      else if (p % 4 == 1) {
        int PP = p - 1;
        for (i = 0; i < n; i++) {
            for (j = 0; j < m; j++) {
                aux = 0;
                for (k = 0; k < PP; k += 4) {
                    aux += A[i][k]*B[k][j] + A[i][k+1]*B[k+1][j] +
                           A[i][k+2]*B[k+2][j] + A[i][k+3]*B[k+3][j];
                }
                C[i][j] = aux + A[i][PP]*B[PP][j];
            }
        }
    } else if (p % 4 == 2) {
        int PP = p - 2, PPP = p - 1;
        for (i = 0; i < n; i++) {
            for (j = 0; j < m; j++) {
                aux = 0;
                for (k = 0; k < PP; k += 4) {
                    aux += A[i][k]*B[k][j] + A[i][k+1]*B[k+1][j] +
                           A[i][k+2]*B[k+2][j] + A[i][k+3]*B[k+3][j];
                }
                C[i][j] = aux + A[i][PP]*B[PP][j] + A[i][PPP]*B[PPP][j];
            }
        }
    } else {
        int PP = p - 3, PPP = p - 2, PPPP = p - 1;
        for (i = 0; i < n; i++) {
            for (j = 0; j < m; j++) {
                aux = 0;
                for (k = 0; k < PP; k += 4) {
                    aux += A[i][k]*B[k][j] + A[i][k+1]*B[k+1][j] +
                           A[i][k+2]*B[k+2][j] + A[i][k+3]*B[k+3][j];
                }
                 C[i][j] = aux + A[i][PP]*B[PP][j] + A[i][PPP]*B[PPP][j] +
            A[i][PPPP]*B[PPPP][j];
            }
        }
    }

    return C;
}
```